

Computer Experiments

Lin Wang

Motivation

- Expensive / Impossible physical experiments
- Applications: engineering, climate, drug development
- Key idea: experiments on *simulators*
- Mathematical models implemented in computer codes
- Codes can be computationally intensive, solving systems of differential equations, finite element analysis, and more.
- Highly nonlinear response surfaces, complex

Example 1: Drug Dose Optimization

We want to understand how **dose and patient variability** affect drug response.

Real trials are expensive
ethically constrained
Simulator allows a
large virtual population

Simulator:

$$y = f(\text{Dose}, CL, V, k_a)$$

- Dose: treatment level
- CL : clearance
- V : volume
- k_a : absorption rate

Inputs (4D space):

- Dose $\in [10, 200]$
- CL $\in [0.5, 2.0]$
- V $\in [10, 50]$
- $k_a \in [0.1, 2]$

y: AUC (Area Under the Concentration) Higher AUC $\rightarrow$ more overall exposure
Peak Concentration, highest concentration reached after dosing

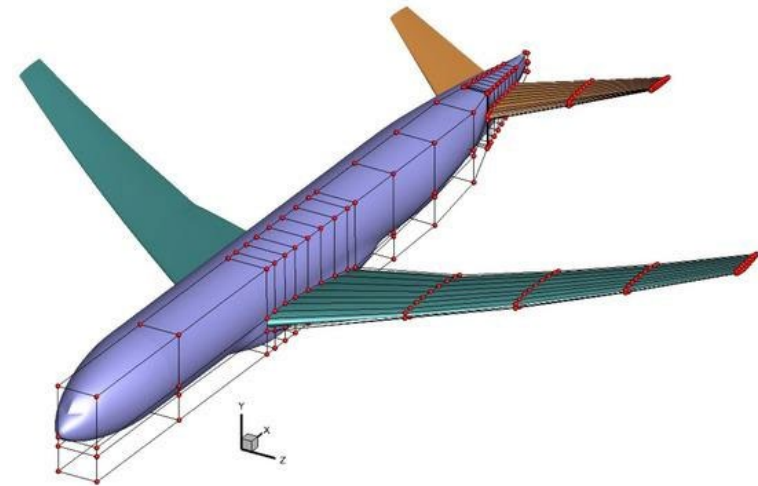
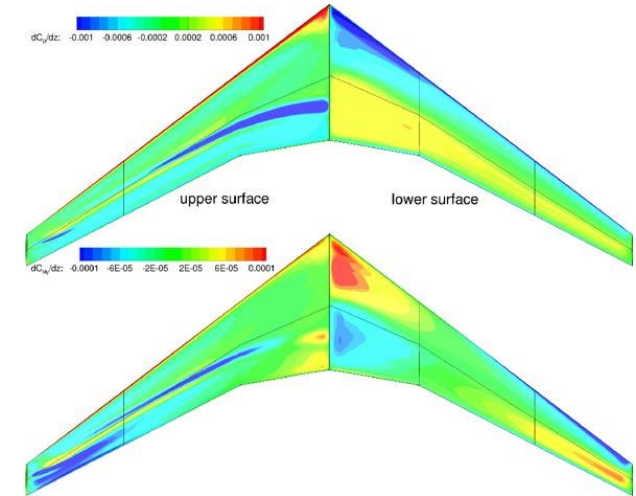
Example 2: Aircraft Wing Design

Simulator inputs:

- wing span
- thickness
- curvature
- angle of attack

Output:

- drag coefficient



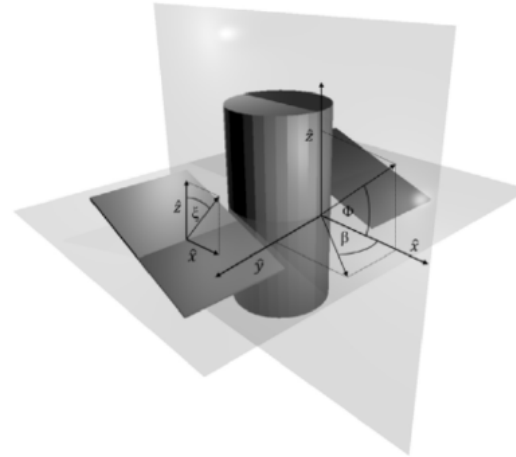
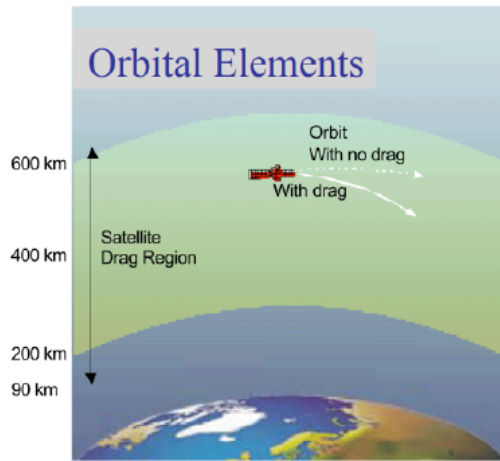
Example 3: Digital Twin/ Virtual Patient

We refine your setup:

$$y = f(a, x)$$

- a : treatment decision (dose, regimen, device setting)
- x : patient features (covariates)
- y : outcome (scalar or functional)

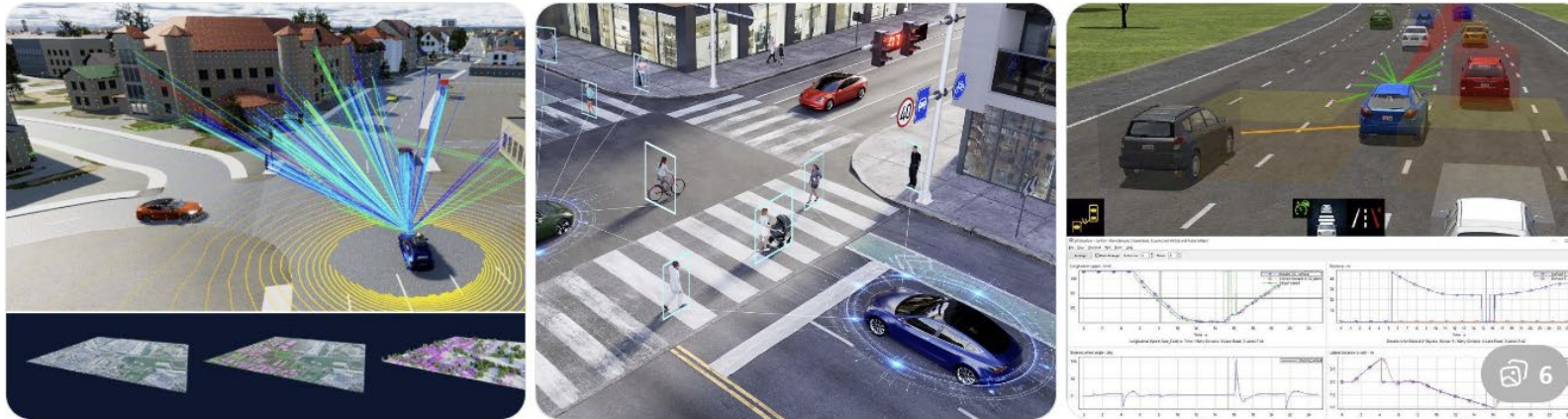
Example 4: Predicting Satellite Drag



- Determining the drag is crucial in positioning and collision avoidance.
- Drag depends on geometry, orientation, ambient and surface temperatures, and atmospheric chemical composition in LEO, which depends on the position: latitude, longitude, and altitude.
- LANL (Los Alamos National Lab) developed the TPMC simulator in C, which takes around 15 mins to do a single run and obtain a response, too slow for real-time prediction and decision-making
- Data collection may be even slower for an experiment involving hundreds of input variables

Example 5: Autonomous Driving

Self-Driving Car Simulation



Model

$$y = f(\text{speed, steering, environment})$$

- Inputs:
 - control parameters
 - road conditions
- Output:
 - safety metrics
 - collision probability

Computer Experiment

- cannot test in real world safely
- simulate thousands of scenarios

Key Differences

Aspect	Physical Experiment	Computer Experiment
Process	Real physical system	Computer simulator or numerical model
Variability	Measurement error and natural variation often present	Often deterministic for fixed inputs, though some simulators are stochastic
Replication	Used to estimate experimental error	Usually unnecessary for deterministic codes, but useful for stochastic codes
Design priority	Treatment comparison, precision, and error control	Space-filling, prediction, and efficient exploration
Main objective	Estimate effects, compare settings, optimize process	Emulate ($f(x)$), optimize inputs, and assess sensitivity

What does it mean to “emulate $f(x)$ ”?

We have a simulator:

$$y = f(x)$$

- $x \in \mathbb{R}^d$: input (design variables)
- $f(x)$: expensive or slow to evaluate

Key idea

We build a **statistical approximation**:

$$\hat{f}(x) \approx f(x)$$

This $\hat{f}(x)$ is called an **emulator** (or surrogate model)

Emulating $f(x)$ means:

Learn a fast, accurate approximation $\hat{f}(x)$ using a limited number of runs.

How we build the emulator

1. Choose design points:

$$x_1, x_2, \dots, x_n$$

2. Run simulator:

$$y_i = f(x_i)$$

3. Fit model:

$$\hat{f}(x) \text{ using } \{(x_i, y_i)\}$$

Why we need an emulator

- Simulator is:
 - expensive (minutes to hours per run)
 - typically not interpretable
- Emulator is:
 - fast to evaluate
 - smooth and continuous
- Emulator enables:
 - prediction at new x
 - optimization
 - sensitivity analysis

Surrogate modeling

To establish the response surface:

- Low-order polynomials, first-order model

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon$$

- Low-order polynomials, second-order model

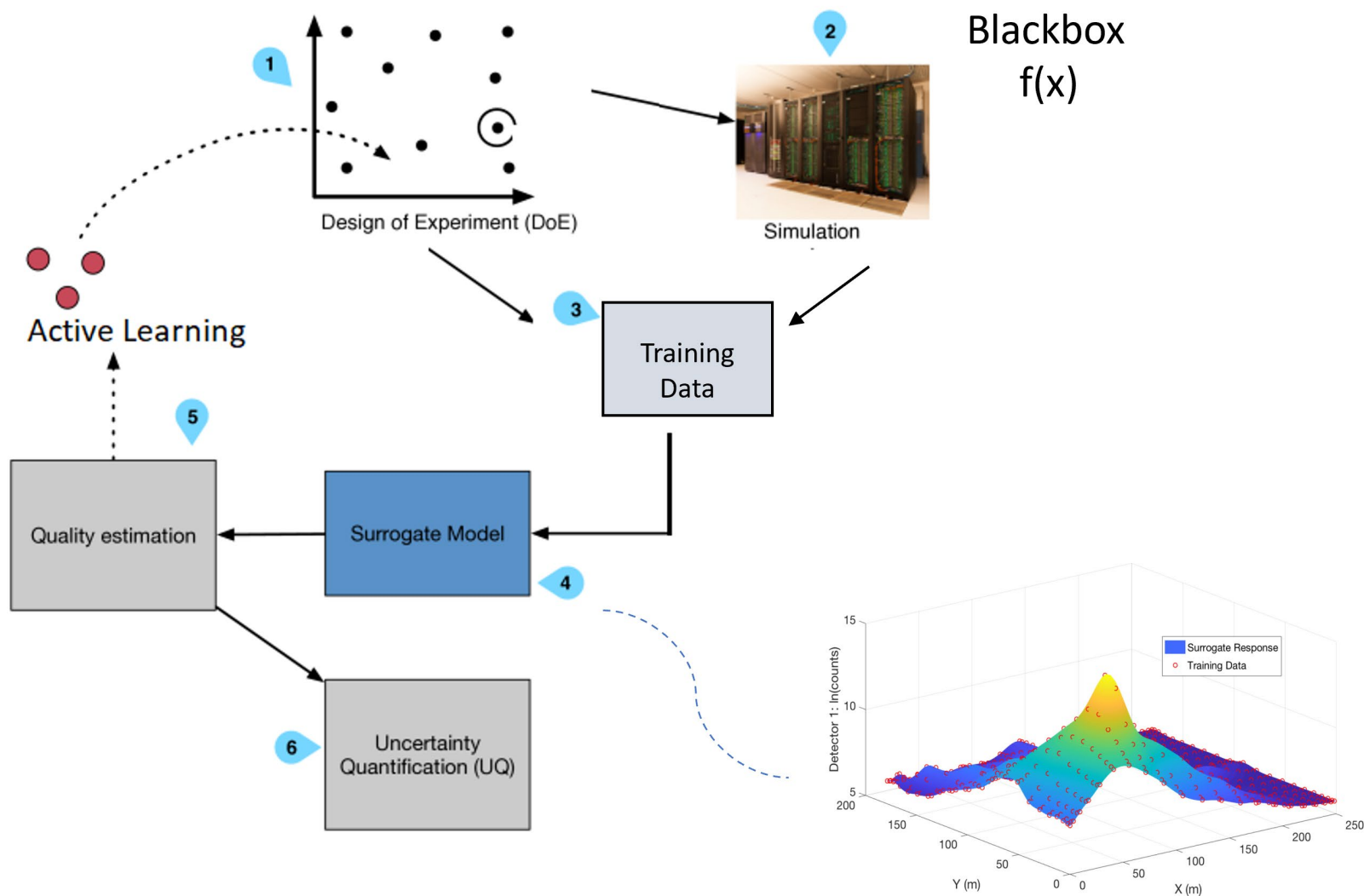
$$y = \beta_0 + \sum_{j=1}^p \beta_j x_j + \sum_{j=1}^p \beta_j^2 x_j^2 + \sum_{j \neq k} \beta_{jk} x_j x_k + \varepsilon$$

- Highly nonlinear: Gaussian process model

$$y(x) = \mu + Z(x) + \varepsilon$$

where $Z(x)$ is a stationary Gaussian process with zero mean and covariance function defined by a kernel.

Procedure for Studying Computer Experiments



Classical design fails here

- Assume linear or low-order models, but a simulator may be highly nonlinear and high dimensional
- Factorial design size grows exponentially
- Different modeling objectives

Design Goal in Computer Experiments

- **Objective:** minimize prediction error over the whole space
- High signal-to-noise ratios so that replication and randomization are typically unnecessary

We want to **cover the input space well**

- Avoid clustering
- Avoid gaps
- Explore entire domain

Space-filling Design

Space-filling design strategy: to seek design points that fill a design region as uniformly (or evenly) as possible

- encourages a diversity of data and ultimately yields the best fitted model, leading to more accurate predictions at out-of-sample testing locations
- model-free

